A determined witness for intuitionistic primitive recursive arithmetic

Removing the introduction witness without a naming exception

Research report prepared for Vladimir Reshetnikov

12 November 2026

Abstract

The native scalar calculus in the current ConciseIPRA proposal introduces $D_{x,y}B$ by supplying an arbitrary numerical term as witness. That choice can be removed. Always use the newest free numerical parameter, and use 0 only when the numerical scope is empty. Keep the separate progress rule for literal equations. The resulting raw calculus proves exactly the same open sequents as the existing kernel, with the identity translation on terms and formulas. Consequently its ordinary realization remains equiconsistent and bi-interpretable with full first-order iPRA, with both numerical domains whole and both comparison graphs $x = y$.

The key construction obtains a fresh name for any term through numerical cases and successor injectivity. No special naming-pattern test and no normalizer is needed. A careful proof uses procedures that rebuild proofs under additional numerical parameters; it does not assume the structural closure it is trying to establish. The same reduction applies to the guarded clause calculus. Its matched complete strict checker has 1,070 significant Python tokens, 1,243 AST nodes, and 3,425 normalized bytes, versus 1,078/1,252/3,442 for the unrestricted control and 1,159/1,339/3,688 for the previous deterministic proposal.

This is a proved local simplification, not a theorem of absolute minimality. The new equivalence is a written proof. The archive includes executable checkers, checked symbolic proof packets, arithmetic checks, and the reproducible comparison; it does not claim a new Lean verification.

Contents

1 The baseline and the precise improvement **2**

2 A complete raw axiomatization **2**

3 Eliminating the introduction witness **4**

4 The witness-shape question and its exact limit **8**

5 Why this is full iPRA, including both interpretations **8**

6 A measured six-family companion on guarded formulas **10**

7 Executable evidence and reproducibility **14**

A Predecessor in the fragment without witnessed introduction **15**

B Internal scalar predecessor bootstrap **16**

1 The baseline and the precise improvement

The comparison is pinned to the ConciseIPRA directory at commit `8fa5ade009`, retrieved on 12 November 2026 [1]. Its README lists eighty-four research packages. The native eight-rule calculus is defined by `NPrf`; the existing Lean development proves its equivalence and formula round trips with iPRA [2, 3]. The README's Further question 2 asks whether case-introduction witnesses restricted to 0 and variables are sufficient. The present theorem answers that question affirmatively and eliminates even the choice between available variables.

The improvement is an actual restriction of a rule on unchanged raw syntax. The terms supplied as introduction witnesses previously ranged over all finite scalar terms. In the proposed rule there is no supplied witness, no witness-shape check, and no naming-pattern exception. The scope alone determines the instance. The proof checker still checks capture avoidance, scopes, formula grammar, every premise, and the final conclusion. No computation or proof search has been hidden in a trusted helper.

The distinction between mathematical simplicity and source-code compression is essential. Some existing compact codecs are much shorter than the strict implementations compared here. We do not claim to beat their unrelated global byte records, or to prove that a simpler theory cannot exist. The measured reduction concerns the same strict grammar, certificate format, error handling, and arithmetic basis. The mathematical contribution is the elimination of a previously required witness choice.

2 A complete raw axiomatization

2.1 Terms, formulas, and numerical scopes

There is one numerical sort. Terms are the finite expressions

$$t ::= x \mid 0 \mid S(t) \mid I(n, a; u.h).$$

Only u is bound in h; the counter n and initializer a are outside its scope. Other variables of h are passive numerical parameters. An iterator is a finite term description, not a code passed to an object-language universal evaluator. Terms are raw syntax: no equation is built into syntactic equality and there is no normalization convention.

Formulas are

$$A ::= (s = t) \mid Q_y A \mid D_{x,y} A.$$

The two variables bound by D are distinct. Their ordinary readings are

$$\begin{aligned} Q_y B(y) &:= B(0) \to \forall y\, B(y), \\ D_{x,y} B(x,y) &:= \exists x (B(x,0) \to \forall y\, B(x,y)). \end{aligned}$$

These readings explain the interpretation; ordinary connectives and quantifiers are not additional primitives of the native proof language. The designated native contradiction is $0 = S0$.

A judgment has an ordered numerical parameter environment $\Pi = (a_1, \ldots, a_n)$, a finite list Γ of formulas in that environment, and a conclusion A. Write $\Pi; \Gamma \vdash A$. The order is part of the proof state; unused parameters are permitted. Define

$$\ell(\Pi) = \begin{cases} a_n, & n > 0, \\ 0, & n = 0. \end{cases}$$

A newly introduced eigenparameter is appended to Π. All substitution is capture avoiding. Bound variables are freshened when needed. Below, Π is suppressed except where it determines a witness.

2.2 Exactly eight primitive rule families

The following is the entire primitive inventory. Computation equations are axiom schemas and may be used in any context.

1. Hypothesis. If A occurs in Γ, then $\Gamma \vdash A$.

2–3. Iterator computation. For every well-scoped n, a, h,

$$\Gamma \vdash I(0, a; u.h) = a, \qquad \Gamma \vdash I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0, IS}$$

4. Crossed transport in one term. For any numerical term template $h(v)$,

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash h(s) = r}{\Gamma \vdash r = h(t)}. \tag{Tr1}$$

5. Closure elimination. For any term t,

$$\frac{\Gamma \vdash Q_y B(y) \qquad \Gamma \vdash B(0)}{\Gamma \vdash B(t)}. \tag{QE}$$

6. Witness elimination.

$$\frac{\Pi; \Gamma \vdash D_{x,y} B(x, y) \qquad \Pi, b; \Gamma, Q_y B(b, y) \vdash A}{\Pi; \Gamma \vdash A}. \tag{DE}$$

Here b is fresh for Π, Γ, B, A. It may occur only in the displayed local branch and is absent from the exported conclusion.

7. Determined case introduction.

$$\boxed{\frac{\Pi, z; \Gamma, B(\ell(\Pi), 0) \vdash B(\ell(\Pi), Sz)}{\Pi; \Gamma \vdash D_{x,y} B(x, y)}}. \tag{DI*}$$

Here z is fresh for the outer environment, context, and all fixed parameters of B. The witness is selected *before* z is added. There is no term argument in the inference certificate.

8. Equation progress. For a literal equation $E(y)$ only,

$$\frac{\Pi, z; \Gamma, E(z) \vdash E(Sz)}{\Pi; \Gamma \vdash Q_y E(y)}. \tag{QI}$$

The parameter z is fresh for the outer context and all passive parameters of E. A formula under a Q or D binder is not a literal equation and cannot be used as an induction invariant here.

There are no additional primitive logical axioms, equality axioms, successor axioms, structural rules, or induction schemas. For example, reflexivity follows from I0 and Tr1: transport the equation $I(0, t; u.u) = t$ along itself with the identity template. Equation induction is QI followed by QE with a base proof. Full formula transport, ordinary logical rules, numerical substitution, and cut are derived later. This explicit boundary is necessary because DI* is not preserved by merely appending an unused numerical parameter to a proof tree.

3 Eliminating the introduction witness

Write $\mathcal{N}$ for the pinned eight-rule native calculus and $\mathcal{K}$ for the calculus in which case introduction always chooses the newest numerical parameter, or 0 in an empty numerical scope. The two systems have exactly the same terms, formulas, contexts, and ordinary readings. In particular, $\mathcal{K}$ retains the separate progress rule for literal equations. The result in this section does not require merging the two introduction modes.

Theorem 3.1 (Exact equivalence on open sequents)**.** *For every numerical scope, every finite native context Γ, and every native formula A,*

$$\Gamma \vdash_{\mathcal{K}} A \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} A.$$

The forward translation preserves every proof instruction except that it supplies the witness selected by $\mathcal{K}$. The reverse translation is effective and preserves every term and formula of the end-sequent.

3.1 Why a proof must be rebuilt under a new parameter

The selected witness depends on the numerical scope, including parameters that do not occur in the conclusion. Consequently, adding a fresh parameter to an existing proof tree can change the witness required by an introduction instruction. Numerical weakening is therefore not an immediate operation on $\mathcal{K}$ proof trees. The usual proof of cut also cannot be assumed: when the receiving proof enters a fresh-parameter branch, the proof being inserted would have to be weakened numerically.

We use *persistent proof builders*. Such a builder for $\Gamma \vdash A$ supplies, for every extension of the numerical scope, a newly constructed $\mathcal{K}$ proof of the shifted A from the shifted Γ. A builder may inspect the extended scope and generate its own fresh parameters. It does not shift a previously constructed proof tree. All builders below are uniform in the displayed term and formula arguments.

Context weakening at a fixed numerical scope is admissible by induction on proofs: adding formula assumptions does not change the selected numerical witness, either at the root or inside a branch. This gives the following limited cut principle before general cut has been established.

Lemma 3.2 (Persistent cut)**.** *A persistent builder of $\Gamma \vdash A$ may replace an assumption A in any $\mathcal{K}$ derivation from Γ, A. If that second derivation is also supplied persistently, the result is persistent.*

Proof. Traverse the second proof tree. At an occurrence of the replaced assumption below k fresh numerical parameters, call the first builder at extension k and context-weaken its result by the local assumptions. Every retained instruction has exactly its former numerical scope, so every selected witness is unchanged. For a persistent receiving proof, perform this traversal anew at each requested ambient extension. □

We will use this lemma only with the explicitly uniform builders constructed below. It does not assert that every $\mathcal{K}$ derivation is already persistent.

3.2 Closure introduction and formula transport

The selector term is

$$C(v, a, b) = I(v, a; u.b), \qquad C(0, a, b) = a, \qquad C(Sv, a, b) = b.$$

The unused update variable in this definition is intentional.

Lemma 3.3 (Uniform closure introduction)**.** *The following rule is admissible, with z fresh:*

$$\frac{\Gamma, B(0) \vdash_{\mathcal{K}} B(Sz)}{\Gamma \vdash_{\mathcal{K}} Q_y B(y)}.$$

Its construction works in every numerical scope. Hence a base proof $\Gamma \vdash B(0)$ and a successor proof $\Gamma \vdash B(Sz)$ give $\Gamma \vdash B(t)$ for any term t.

Proof. Introduce $D_{x,y}B(y)$, where x has no occurrence. Substituting the automatically selected witness for x therefore leaves the premise unchanged. Eliminate this D formula: its opened assumption is precisely $Q_y B(y)$, independent of the fresh witness, and that assumption may be exported. For the last assertion, context-weaken the successor proof by $B(0)$, apply the first assertion, and use QE at t.

When this construction is used inside a builder, its successor proof is generated with the z fresh in that invocation. There is no assertion that a fixed successor proof can be arbitrarily substituted or weakened. □

Lemma 3.4 (Uniform Leibniz rule)**.** *For every native formula template $F(v)$ and terms s, t, there is a persistent builder of*

$$s = t,\ F(s)\ \vdash_{\mathcal{K}}\ F(t).$$

Consequently persistent proofs of $s = t$ and $F(s)$ give a persistent proof of $F(t)$ over the same context.

Proof. Induct on the formula template. For an equation, the crossed one-term transport rule gives the ordinary equality operations. Transport in the left and right term templates takes two applications. These are uniform finite proof recipes.

For $F(v) = Q_y B(v, y)$, retain $s = t$ and $Q_y B(s, y)$ as assumptions. To introduce the target closure, assume $B(t, 0)$ in its fresh-z premise. The induction hypothesis, with the equality reversed, gives $B(s, 0)$. QE applied to the retained source closure gives $B(s, Sz)$, and the induction hypothesis gives $B(t, Sz)$. Lemma 3.3 finishes the construction. Each matrix transport has smaller formula complexity; its uniform builder can be used at the current terms and scope. Compositions use Lemma 3.2.

For $F(v) = D_{u,y}B(v, u, y)$, eliminate the retained source formula at a fresh parameter b. Its opened assumption is $Q_y B(s, b, y)$. Introduce the target $D_{u,y}B(t, u, y)$ in this context. The parameter b is now newest, so it is exactly the witness required by the rule. Under $B(t, b, 0)$, the matrix induction hypothesis transports to $B(s, b, 0)$; QE on the opened source closure gives $B(s, b, Sz)$; the matrix induction hypothesis transports back to $B(t, b, Sz)$. The target D formula does not contain b, so DE discharges it.

In every invocation after an ambient scope extension, a new b is generated *after* all ambient parameters. Thus the same recipe still introduces at the newest parameter. This proves persistence without numerically weakening a proof containing an old b. The final assertion follows by persistent cut. □

3.3 The arithmetic dependency

The only arithmetic fact needed for naming is a scalar predecessor term P with a uniform proof of

$$P(0) = 0, \qquad P(Sr) = r. \tag{1}$$

The scalar bootstrap supplied with the comparison material constructs such a finite term using $0, S, I$ alone. Its resources are computation, equality, induction on a literal equation, and numerical cases with equation conclusions. The accompanying arithmetic verification explains

how to carry that bootstrap out before the witness-elimination theorem. In particular it does not appeal to general existential introduction.

For completeness, even conditional equation cases can be implemented uniformly at this stage. For equations $E \equiv p = q$ and $F \equiv r = s$, define

$$E \Rightarrow_{\mathrm{eq}} F := Q_v(C(v, p, r) = C(v, q, s)).$$

Its introduction rule follows from Lemma 3.3, because the zero row computes to E and every successor row computes to F. Its elimination rule is QE at $1 = S0$, followed by computation. Now suppose a numerical case argument derives F under $e = 0$, and under $e = Sz$ with z fresh. Apply these equation-implication rules to

$$H(y) := (e = y) \Rightarrow_{\mathrm{eq}} F.$$

The two branches give $H(0)$ and $H(Sz)$. Clean numerical cases give $H(e)$, which applied to reflexivity $e = e$ gives F. With branch builders this is a persistent construction, including arbitrary retained side assumptions. Thus contextual equation cases in the written scalar bootstrap create no dependence on the theorem being proved.

Applying P to an equation $Ss = St$, and using (1), gives the persistent rule

$$Ss = St \ \vdash_{\mathcal{K}} \ s = t. \tag{2}$$

No naming packet is used to establish this rule.

3.4 A fresh name for every term

Lemma 3.5 (Naming without choosing a witness)**.** *Let*

$$X(y) := D_{x,v}(C(v, 0, C(y, 0, Sx)) = C(v, 0, y)). \tag{3}$$

For every term t, $X(St)$ has a persistent proof. When this formula is opened at a fresh parameter b, its retained closure yields a persistent proof of $b = t$.

Proof. First prove $X(0)$. Whatever witness w the current scope selects, the case-introduction successor premise computes to $C(0, 0, Sw) = 0$, hence to $0 = 0$. The assumption in that premise is unneeded. Therefore $X(0)$ is available even in an empty scope, and also when the selected witness is an unrelated ambient parameter.

For $X(Sz)$, generate the numerical-case parameter z freshly. It is the newest parameter when the inner D is introduced, so the rule selects $x = z$. The inner successor premise computes to $C(Sz, 0, Sz) = Sz$, hence to $Sz = Sz$. Again the premise's base assumption is unneeded. Lemma 3.3 now gives $X(r)$ for an arbitrary target term r; take $r = St$.

Open this packet at a fresh b. Its closure has body

$$C(v, 0, C(St, 0, Sb)) = C(v, 0, St).$$

At $v = 0$ the body is a computational reflexivity. QE at 1, followed by selector computation, gives $Sb = St$. Equation (2) gives $b = t$.

All these proofs are rebuilt uniformly at each ambient scope. In particular the z used to prove the successor case is always generated after the current parameters. The last alias proof uses only the retained closure, QE, computation, equality and the predecessor builder, so it remains persistent under further parameters. □

The ordinary reading of $X(y)$ is $\exists x\,[C(y, 0, Sx) = y]$. The argument uses this formula at St, where the selector computes away, and then uses injectivity. It never attempts to introduce $\exists x\,(x = t)$ directly with the prohibited witness t.

3.5 The protected compiler

Proof of Theorem 3.1. The forward direction is immediate: every restricted introduction is an instance of the old introduction rule. For the converse, strengthen the induction statement on an old derivation p of $\Gamma \vdash A$. The compiler accepts:

1. an arbitrary term substitution σ from the source numerical scope into an arbitrary target scope;
2. a target context Δ;
3. for each $E \in \Gamma$, a persistent Δ-proof builder of $E[\sigma]$.

It returns a persistent Δ-proof builder of $A[\sigma]$. To answer a request at an extended target scope, it reruns its construction with shifted σ and the supplied hypothesis builders. It does not shift its previously returned proof.

Hypothesis, computation, one-term transport and QE are handled directly. For a source QI-progress instruction, generate its fresh target parameter, extend σ by that parameter, and invoke the induction hypothesis on the step subderivation. Supply the old hypothesis builders in the extended scope and the persistent literal-hypothesis builder for its equation assumption. Apply QI-progress. Source DE is treated in the same manner: open its translated major premise at a fresh parameter, extend σ, and translate the minor subderivation using the opened closure as a new persistent hypothesis. Every eigenvariable condition is preserved.

Consider source case introduction with arbitrary witness t and matrix $B(x, y)$. Put $t' = t[\sigma]$ and let B' be the matrix obtained by substituting for its ambient free parameters. Build $X(St')$ by Lemma 3.5 and open it at fresh b. From the retained closure obtain the persistent alias $b = t'$.

We now introduce $D_{x,y}B'(x, y)$. At this point b is the newest parameter, so the target rule selects it. In the rule's fresh-z premise the local hypothesis is $B'(b, 0)$. Formula transport gives a persistent proof of $B'(t', 0)$. Invoke the induction hypothesis on the source case-premise subderivation, using σ extended by z. Its original hypotheses are represented by the inherited builders; its source base hypothesis is represented by this transport builder. It returns $B'(t', Sz)$. Transport back along $b = t'$ to obtain $B'(b, Sz)$, apply the restricted introduction, and discharge b using DE on the naming packet. The exported formula does not contain b.

This construction is well founded. Every call to the proof compiler uses a strict subderivation of p. Formula transport uses induction on formula complexity. The arithmetic and naming builders were constructed before this compiler, without invoking its equivalence theorem. Persistent hypotheses, rather than unproved numerical weakening or general cut, provide all premises needed under new scopes.

Finally use the identity substitution and literal-hypothesis builders. The output is a $\mathcal{K}$ proof of the original open sequent. □

Corollary 3.6 (Structural rules and the ordinary theory)**.** *The new calculus admits numerical substitution, numerical weakening and cut. Its ordinary realization is equiconsistent and bi-interpretable with iPRA, with both numerical domains whole and both numerical round-trip graphs equal to $x = y$.*

Proof. For the structural rules, translate to $\mathcal{N}$, use its established structural rule, and translate back. These rules are consequences of the equivalence, not premises of its proof.

Between $\mathcal{K}$ and $\mathcal{N}$ the interpretations are literally the identity on terms and native formulas. Compose this identity equivalence with the pinned native-to-ordinary and ordinary-to-iPRA interpretations. The ordinary meanings of Q and D, and the definitions of the ordinary intuitionistic connectives, are unchanged. Their existing formula round trips therefore apply unchanged. The numerical maps also remain unchanged, so their composites retain the

identity graph $x = y$. The designated contradiction is still $0 = S0$; its correspondence with ordinary falsity and its elimination rule are preserved by exact open-sequent equivalence. Thus consistency is preserved in both directions as well. □

Verification boundary. The argument above is a written metatheorem. The pinned development provides machine-checked results for the old native calculus and its iPRA interpretation; those results are not a machine-checked proof of this new compiler. Executable certificate checks validate their actual finite certificates only. A complete formalization of the new result must encode the persistent builders or an equivalent protected translation, rather than assume numerical weakening of restricted proof trees.

4 The witness-shape question and its exact limit

Let $\mathcal{W}$ be the old native calculus with the witness of case introduction restricted to the literal 0 or a free numerical variable. Let $\mathcal{N}$ and $\mathcal{K}$ denote the old and new calculi. Every DI* witness is allowed in $\mathcal{W}$, and every witness allowed in $\mathcal{W}$ is allowed in $\mathcal{N}$. Thus

$$\mathcal{K} \subseteq \mathcal{W} \subseteq \mathcal{N}.$$

Theorem 3.1 gives the opposite inclusion on derivable sequents. Hence all three consequence relations agree. This is the positive statement left open by `NativeWitnessShape.lean` at the pinned commit [4].

Both shapes are necessary if one restricts the witnesses solely by choosing a subset of $\{0, \text{variables}\}$. With 0 alone, interpret $D_{x,y}B$ as its closure at $x = 0$. The other rules remain sound in the natural numbers, but

$$D_{x,y}(C(y, 1, x) = 1)$$

is false under that reading, whereas the unrestricted rule proves it at witness 1. With variables alone, a closed empty-context derivation cannot produce a first D formula. More formally, when the numerical scope is empty and the context contains only equations and closures of equations, every derivable formula has one of those two forms; induction on the rules proves this invariant. Yet the native kernel proves $D_{x,y}(0 = 0)$. The repository already formalizes both negative statements [4].

This is a small optimality result about the two witness shapes. It is not an optimality result about all proof calculi. It explains why DI* uses 0 at the closed root and why the presence of fresh numerical parameters matters.

5 Why this is full iPRA, including both interpretations

Exact native consequence equivalence is stronger than agreement on standard numerical values or on closed equations. The existing interpretations between the native kernel and iPRA can be reused without changing a single term or formula. This section states those interpretations explicitly enough to identify the transferred theorem.

5.1 The ordinary arithmetic realization

Elaborate each fixed iterator description into a function symbol. For a previously defined term $h(u, \vec{p})$, its symbol i_h obeys

$$i_h(0, a, \vec{p}) = a, \qquad i_h(Sn, a, \vec{p}) = h(i_h(n, a, \vec{p}), \vec{p}).$$

This is a staged first-order signature of finite descriptions. Let T_I have intuitionistic first-order equality logic over this signature, the displayed computation equations, contextual induction for literal equations, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The last implication is an explicit bridge to ordinary falsity. Merely listing positive arithmetic equations would allow a singleton structure; we do not identify an equation with ordinary falsity without this bridge.

The interpretation of native Q, D is their displayed ordinary reading. Its domain formula is $x = x$, and interpreted equality is actual equality. DI* is sound because it is an instance of the old case rule; coverage justifies that old rule. QI is sound by equation induction.

5.2 Ordinary logical connectives as finite native macros

For a convenient description use the equivalent D-only notation, replacing $Q_y B$ by $D_{u,y} B$ with unused u. Set $C(v, a, b) = I(v, a; u.b)$. Gate every bottom equation by

$$G_v(s = t) = (C(v, 0, s) = C(v, 0, t)), \qquad G_v(D_{x,y} B) = D_{x,y} G_v B.$$

All bound variables are fresh for the control v. Then $G_0 A$ is provable, $G_{Sw} A$ is equivalent to A, and $D_{u,y} G_y A$ is a padding of A by one unused existential coordinate.

A selector $M_v(A, B)$ between arbitrary formulas is defined externally by finite structural recursion. At equations it is

$$M_v(s = t, r = q) = (C(v, s, r) = C(v, t, q)).$$

For two D formulas, align their bound variables and apply M to their matrices under one D. If only one side has a D root, pad the equational side with G_y before descending. Each call decreases the maximum binder depth. The selector has endpoint equivalences $M_0(A, B) \dashv\vdash A$ and $M_{Sw}(A, B) \dashv\vdash B$.

Put $\delta(v) = C(v, 1, 0)$ and $p(z) = I(z, 0; u.\delta(u))$. The following finite macros are the inherited ordinary logical interface [5]:

$$\begin{aligned}
A \Rightarrow_* B &= D_{u,z} M_z(A, B), \\
A \wedge_* B &= D_{u,z} G_z M_{p(z)}(A, B), \\
\exists_* x\, A(x) &= D_{x,y} G_y A(x), \\
A \vee_* B &= \exists_* x\, M_x(A, B), \\
\forall_* x\, A(x) &= A(0) \wedge_* D_{u,z} G_z A(z), \\
\bot_* &= (0 = S0).
\end{aligned}$$

The coordinate u is unused. The conjunction extracts its two operands at $z = 1, 2$, using $p(1) = 1$ and $p(2) = 0$. The universal macro combines the zero instance with all successor instances. Implication and existential introduction use the selector and gate endpoints. These macros have the ordinary introduction and elimination rules and both formula round trips in the existing kernel; exact consequence equivalence transfers their derivations to $\mathcal{K}$.

In particular, the theorem is not a reduction to classical arithmetic or to the quantifier-free fragment. Quantified intuitionistic formulas and their proof rules are recovered through these finite syntactic translations. Progress remains restricted to equations.

5.3 Arithmetic translations and internal round trips

Interpret every fixed scalar iterator in iPRA by primitive recursion whose update ignores the counter argument. Conversely, the existing scalar construction first establishes predecessor, then an equality test reflecting actual equality, then quantifier-free induction, and then pairing with projections. This dependency order avoids assuming pairing or equality reflection in the proof of predecessor.

Write the internally proved pairing and projections as π, L, H. For a fixed primitive recursion

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}),$$

its scalar compilation is

$$\begin{aligned} V(n,\vec{p}) &= I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))\big),\\ f^*(n,\vec{p}) &= H(V(n,\vec{p})). \end{aligned}$$

Equation induction proves $L(V(n,\vec{p})) = n$; the projection identities then verify the defining equations. Compilation is external recursion on each fixed finite description. There is no universal PR evaluator in the object theory.

Both composites are justified internally. Equality induction proves uniqueness of an iterator, and similarly uniqueness of each fixed primitive recursion, from its initial equation and update. An external induction on the finite function description shows that its double translation has the same equations and hence the same values in the theory. This proves preservation on all elements of every model, rather than merely on standard numerals. The whole numerical domain and actual equality are unchanged, so the comparison isomorphism in each direction has graph $x = y$. Structural induction on formulas, using the logical macros and the Q, D readings, gives both formula composites.

The pinned `NativeBiInterpretation.lean` establishes the following statements [3]:

- Faithfulness: `native_iff_pra` and `pra_iff_native`.
- Formula composites: `native_roundTrip` and `pra_native_roundTrip`.
- Equiconsistency: `native_inconsistent_iff`.

The new part is the identity-language equivalence of Section 3; composing it with those established translations proves the required bi-interpretability and equiconsistency. There is no claim that a theory proves its own consistency.

6 A measured six-family companion on guarded formulas

The witness elimination also applies to the repository's strict guarded clause presentation [7]. This companion provides the matched executable size comparison. It uses raw terms and ordinary capture-avoiding substitution: there is no computation normalizer in its trusted syntax.

6.1 Exact language and inference rules

Terms are 0, numerical variables, $S(t)$, and $I(n, a; u.h)$, with u bound only in h. Write

$$C(v,a,b) = I(v,a;u.b), \qquad H = (I(0,0;u.0) = 0).$$

The state variable of C is unused. A formula is an equation or $\mathsf{D}_{x,y}B$. Every immediate equation matrix under a D must have the literal guarded shape

$$I(y,0;u.h) = I(y,0;u.k). \tag{G}$$

The bodies h, k may contain x, y and passive numerical parameters. An equation at the outermost formula level need not be guarded. A compound matrix under D is permitted when its own nested paired formulas satisfy the same grammar. In particular, this is a restriction of the public formula language, not a claim of completeness for arbitrary unguarded paired formulas.

A context retains clauses $[A; v.B(v)]$, read as $A \to \forall v B(v)$, with v absent from A. An ordinary assumption A means the constant clause $[H; v.A]$. The environment is an ordered finite list of numerical parameters. Let ℓ be its newest member, or 0 when that list is empty. The six inference families are the following:

1. The zero computation $I(0, a; u.h) = a$.
2. The successor computation $I(Sn, a; u.h) = h[I(n, a; u.h)/u]$.
3. *Use*: from A, and a retained clause $[A; v.B(v)]$, infer $B(t)$ at any well-scoped numerical term t.
4. *Transport*: from $s = t$ and $F(s)$ infer $F(t)$, for any legal one-place formula template F.
5. *Open*: from $\mathsf{D}_{x,y}B$ and a derivation of A under the additional clause $[B(b, 0); y.B(b, y)]$, infer A. Here b is fresh and absent from the exported conclusion.
6. *Introduce*: infer $\mathsf{D}_{x,y}B$ from

$$\Gamma, B(\ell, \eta) \;\vdash\; B(\ell, Sz), \qquad \eta = \begin{cases} z, & B \text{ is a literal equation,} \\ 0, & B \text{ is compound,} \end{cases} \tag{GI}$$

   where z is fresh. The choice of ℓ is made in the outer environment, before z is adjoined.

There is no supplied introduction witness and no naming-pattern exception. The rule's equation/case selector and the guard grammar are unchanged from the matched source calculus. The designated contradiction is $0 = S0$.

The source calculus $\mathcal{G}$ has these same rules, except that Introduce may choose an arbitrary supplied term t instead of ℓ. Call the new calculus $\mathcal{G}_{\text{new}}$. Their ordinary formula reading is identical:

$$\mathsf{D}_{x,y}B \quad \mapsto \quad \exists x\big(B(x, 0) \to \forall y B(x, y)\big).$$

6.2 A fragment insensitive to the selected witness

Call an introduction *dummy* if its existential coordinate x is absent from $B(x, y)$. Its premise is then independent of the selected witness. Derivations using dummy introductions only can be rebuilt under arbitrary substitutions of numerical terms, injective renamings, and extensions of the numerical parameter environment. Indeed, the deterministic witness may change but its matrix instance does not; the other rules commute with the corresponding substitutions after their local eigenparameters are freshened. This observation applies to this fragment only. It is not an assumption of arbitrary parameter weakening for $\mathcal{G}_{\text{new}}$.

For an equation $A = (s = t)$ define

$$\mathrm{Pad}(A) = \mathsf{D}_{r,v}\big(C(v, 0, s) = C(v, 0, t)\big), \qquad r \text{ unused.}$$

This is a legal guarded formula. From A one derives $\mathrm{Pad}(A)$ by a dummy progressive introduction: at a successor the required equation reduces to A, and the progress assumption is unused. Conversely, Open and Use at $1 = S0$ recover A from its padding; the required zero row follows from the zero computation. Thus padding and unpadding are uniform proof builders using dummy introductions only.

If $A = (s = t)$ and $E = (p = q)$ are equations, put

$$\mathrm{Imp}(A, E) = \mathsf{D}_{r,y}\,\mathrm{Pad}(C(y, s, p) = C(y, t, q)), \qquad r \text{ unused.}$$

Its immediate matrix is compound. The zero row is provably $\mathrm{Pad}(A)$ and each successor row is provably $\mathrm{Pad}(E)$. Consequently the ordinary introduction and elimination rules for this encoded implication follow by dummy case introduction, Open, Use, computation, and padding. No arbitrary existential witness is used in those constructions.

For a compound formula $F(y)$, the unused-witness formula $\mathsf{D}_{r,y}F(y)$ directly gives numerical cases. For an equation $E(y)$, apply these cases to $\mathrm{Pad}(E(y))$ and then unpad. In particular, the following hypothetical split is available for every equational conclusion:

$$\frac{\Gamma, t = 0 \vdash E \qquad \Gamma, t = Sz \vdash E}{\Gamma \vdash E}, \qquad z \text{ fresh.} \tag{HC}$$

To see this without assuming numerical disjunction, set $F(y) = \mathrm{Imp}(t = y, E)$. Implication introduction gives $F(0)$ and $F(Sz)$. Compound numerical cases at t give $F(t)$; reflexivity $t = t$ and implication elimination then give E.

6.3 Equation induction and predecessor without active witnesses

Let $E(y)$ be the equation $s(y) = t(y)$, and set

$$G(y) = \big(C(y, 0, s(y)) = C(y, 0, t(y))\big).$$

Suppose a base proof $E(0)$ and a step construction from $E(z)$ to $E(Sz)$ are supplied uniformly in the ambient context. Under $G(z)$, the hypothetical split (HC) yields $E(z)$: at $z = 0$ use the base proof, and at $z = Sw$ compute the two guards. The step gives $E(Sz)$ and hence $G(Sz)$. Dummy progressive introduction therefore proves $\mathsf{D}_{r,y}G(y)$. Open and Use at the desired term a give $G(a)$. A further hypothetical split, using $E(0)$ in its zero branch, gives $E(a)$.

This recovers contextual induction on literal equations using dummy introductions only. The arithmetic constructions in the scalar predecessor bootstrap therefore take place inside this stable fragment. Its statements that the zero test or parity has values 0 and 1 can be used as elimination schemas for equational conclusions: split the argument of the zero test, or split a parity counter and then the argument of the zero test in its successor computation. The bootstrap does not require a disjunction theorem or existential introduction to perform those splits.

It follows that the fixed scalar predecessor term P satisfies

$$P(0) = 0, \qquad P(Sa) = a$$

in the stable fragment. Substitution into these helper derivations is legitimate there. Successor injectivity is obtained afterward by applying P to an equality $Sa = Sb$; it is not assumed in the predecessor construction.

6.4 Names are available without a special naming case

Define the following legal guarded formula, with y free:

$$A(y) = \mathsf{D}_{x,u}\big(C(u, 0, C(y, 0, Sx)) = C(u, 0, y)\big). \tag{N}$$

In any parameter environment, $A(0)$ can be introduced at its automatically selected witness: the successor row reduces to $0 = 0$ whatever that witness is. In the numerical successor branch $A(Sz)$, the parameter z is newest. Rule (GI) therefore selects exactly z, and its successor row reduces to $Sz = Sz$. Both constructions ignore the local progress assumption.

Compound numerical cases now prove $A(Sa)$ for every term a. This is Use at the term Sa, not numerical substitution into a finished proof containing an active variable witness. Open $A(Sa)$ at fresh b and Use its clause at 1. Computation gives $Sb = Sa$, and the predecessor equations give $b = a$.

This construction supplies a name for every term in every actual ambient environment. It uses no unrestricted introduction. The literal naming-pattern dispatch of the earlier deterministic checker is consequently unnecessary.

6.5 The translation respects every local parameter environment

For precision, a *persistent proof builder* recreates its indicated proof after adjoining arbitrary further parameters and retained resources, while fixing the old parameters and resources. A source clause realization is similarly an operation which, in every such extension, turns a proof of its antecedent into a proof of its body at any requested well-scoped term. Retained target clauses have these realizations directly by Use. The stable arithmetic constructions and the naming construction just established are persistent builders.

Induct on a source $\mathcal{G}$ derivation, universally quantified over its numerical substitution into an arbitrary target environment and over persistent realizations of its source clauses. Computation and transport are copied. A Use invokes the corresponding clause realization. At Open, first adjoin the actual fresh target parameter and its retained clause; then invoke the induction hypothesis on the strictly smaller source branch in that environment.

Consider the remaining source introduction, with arbitrary supplied witness a. Build (N) at Sa and open it at fresh b. Extract the equality $b = a$. The parameter b is now newest, so the target introduction selects it. Transport its local assumption $B(b, \eta)$ to $B(a, \eta)$, invoke the induction hypothesis on the strictly smaller source branch with this transported assumption as a persistent builder, and transport the result $B(a, Sz)$ back to $B(b, Sz)$. The mode agrees in source and target: substitution for an outer parameter does not change whether the matrix is an equation or is compound. The resulting $\mathsf{D}_{x,y}B$ contains no b, so the naming Open closes.

Whenever a helper opens another parameter, the source induction hypothesis is invoked in that actual further extension. A completed target proof is never silently weakened to a new numerical scope. Every recursive source call is on a strict subderivation. This establishes the required persistent-builder induction and avoids using target cut or arbitrary parameter weakening before they have been proved admissible.

Theorem 6.1 (Guarded consequence equivalence)**.** *For every ordered numerical parameter environment, every legal clause context* Γ*, and every legal guarded formula* F*,*

$$\Gamma \vdash_{\mathcal{G}} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{G}_{\mathrm{new}}} F.$$

The interpretation is the identity on terms, formulas, and clauses.

Proof. The persistent translation proves the forward direction. Every new introduction is a permitted instance of unrestricted introduction, which proves the reverse direction by inclusion. The other rules are identical. The argument applies to arbitrary legal contexts, not only closed theorems. □

Combining this theorem with the inherited guarded presentation theorem gives equiconsistency and bi-interpretability with iPRA. The numerical domains remain whole, equality remains actual equality, and both numerical comparison isomorphisms have graph $x = y$. The existing formula round trips are unchanged. Arbitrary target parameter weakening, numerical substitution, and cut are now consequences of the equivalence theorem; they were not premises of its proof.

6.6 Matched complete-checker comparison

The delivered strict checker is obtained from the existing deterministic guarded checker by deleting its entire `witness` function and instantiating the matrix directly at `-scope`. At scope zero that integer is numerical zero; otherwise it is the newest negative parameter level. All other rules, the strict public grammar validator, capture-avoiding substitution, eigenparameter checks, JSON handling, and final-conclusion check are unchanged.

The measurement removes actual docstrings, applies `ast.unparse`, and appends one line feed. Tokens are significant Python tokens; AST nodes count the complete parsed normalized module. The same measurement reproduces the published baseline figures.

Complete strict guarded checker	Tokens	AST nodes	Bytes
Unrestricted witness control	1078	1252	3442
Earlier deterministic naming dispatch	1159	1339	3688
Newest parameter, no dispatch	1070	1243	3425

The reduction against the unrestricted control is eight tokens, nine AST nodes, and seventeen normalized bytes. Against the previous deterministic proposal it is eighty-nine tokens, ninety-six AST nodes, and 263 normalized bytes. These are matched measurements of complete strict checkers. They establish a concrete reduction and do not establish an absolute minimum among unrelated compact codecs or among all possible axiomatizations.

7 Executable evidence and reproducibility

The archive contains two standalone checkers. The native checker is a readable implementation of the exact eight rules in Section 2. Its JSON input may specify an open numerical scope and assumptions. The guarded checker is the measured six-family implementation. Each checker reads a certificate, performs all checks locally, and returns `ACCEPT` or `REJECT`; neither imports a theorem library or calls an external prover.

The native suite constructs actual raw proof trees for symbolic numerical coverage and for the opened successor-alias branch. In particular, it checks $X(St)$ with t a free parameter; the proof is not a numerical sample. It also checks the construction in scopes with zero, one, two, and five ambient parameters. The scope-sensitive negative example deliberately reuses a closed proof after adding a parameter; the kernel rejects it, as the mathematical proof predicts. Other negative cases cover eigenparameter escape, invalid bound indices, a supplied witness field, a missing closure base, and compound progress.

The arithmetic suite separately runs 412 checks using the literal scalar definitions, including $P(0), \ldots, P(7)$. It also checks seven symbolic equation-case lemmas through fourteen branches and 127 individually replayed rewrite steps, and rejects a damaged certificate. These checks test the delicate algebra in the predecessor dependency audit. They are not a complete formal proof of the predecessor theorem.

Run `python3 reproduce.py` from any directory. It rebuilds the native certificates, runs the guarded and arithmetic suites, recomputes the matched whole-module metrics, and checks the native checker through its command-line interface. The evidence directory records the exact results. Run `python3 build_pdf.py` to compile this document in three passes with pdfLaTeX. Only the document build requires a TeX installation; the proof checkers and verification scripts use the Python standard library.

Verification boundary. The proof of exact equivalence is a mathematical metaproof in this report. It is not a new theorem already checked by Lean or Rocq. The repository's existing Lean bridge is reused as an established result; it was inspected at the pinned source revision, not rebuilt in this environment. The finite checkers validate their supplied certificates, and

the arithmetic tests validate the stated finite and symbolic instances. The report does not promote those tests to a machine-checked universal completeness theorem.

What remains open. The result removes the introduction witness on raw syntax and resolves the stated witness-shape question. It does not remove equation progress, reduce the raw native rule count, prove an absolute source-size lower bound, or settle the independence of progress. Further compression of the measured checker would require a new matched comparison.

A Predecessor in the fragment without witnessed introduction

This appendix strengthens the dependency statement of the inherited scalar predecessor construction. Its entire proof can be conducted using equations, the two iterator computations, ordinary equality reasoning, the progressive closure rule for literal equations, and closure specialization. In particular, it needs no instance of witnessed introduction. This matters because this fragment is closed under arbitrary numerical substitution, even when the full calculus restricts its existential witnesses to variables and zero.

Write $C(n,a,b) = I(n,a;u.b)$, $Z(n) = C(n,1,0)$, and $a+b = I(b,a;u.Su)$. Let $\mathcal{E}$ denote the fragment just described. An equation-case inference belongs to $\mathcal{E}$: from $E(0)$ and a proof of $E(Sz)$ for fresh z, add the unused assumption $E(z)$ in the latter proof, apply progressive closure, and specialize it using $E(0)$. This inference concerns an equation E; it does not presuppose a disjunction $t = 0 \vee \exists z\, t = Sz$.

The arithmetic equations for a commutative monoid follow by equation induction. Equation cases give the two unconditional identities

$$Z(v) + Z(Z(v)) = 1, \qquad Z(Z(Z(v))) = Z(v).$$

For $N(n) = I(n,0;u.Z(u))$, equation induction consequently gives

$$N(n) + Z(N(n)) = 1, \qquad Z(Z(N(n))) = N(n).$$

Thus every later use of the statement that $N(n)$ is Boolean can be replaced by explicit equations. In particular $N(n+2) = N(n)$, followed by $N(2n) = 0$, $N(2n+1) = 1$, and $N(3n) = N(n)$, uses only equation induction.

The potentially delicate point is the ramp invariant. Put

$$j(u) = Z(u) + 2N(u), \qquad M(n) = I(n,0;u.j(u)), \qquad G(n) = I(n,0;u.u + M(u) + 1).$$

Finite computations give $j(0) = 1$, $j(1) = 2$, and $j(2) = 0$. The following two unconditional equations are established by equation cases on the fresh variable v:

$$M(g + Z(v) + 1) = C(v, j(j(M(g))), j(M(g))),$$
$$C(v, j(j(Z(v))), j(Z(v))) = Z(Z(v)).$$

For the first, the zero branch is the double successor computation for M, and the successor branch is its single successor computation. For the second, the zero branch computes $j(j(1)) = 0$ and the successor branch computes $j(0) = 1$.

Now suppose the single induction equation is $M(G(n)) = N(n)$. Write $g = G(n)$ and $p = N(n)$. Substitute $v = Z(p)$ into the two displayed equations and use $Z(Z(p)) = p$. Ordinary equality reasoning gives $M(g + p + 1) = Z(p)$. The update for G and the current induction equation identify $G(Sn)$ with $g + p + 1$, while $N(Sn) = Z(p)$. This proves $M(G(n)) = N(n)$ by equation induction with no conditional Boolean splitting.

The invariant gives $G(Sn) = G(n) + N(n) + 1$ and $G(n+2) = G(n) + 3$, the latter using $N(n) + Z(N(n)) = 1$. Consequently equation induction proves $G(2n) = 3n$. The successor calculation for $G(3n) = 3G(n) + N(n)$ reduces, with $p = N(n)$, to

$$2p + 4 = 3p + 3 + Z(p).$$

This is the monoid equation obtained from $p + Z(p) = 1$; it requires no disjunctive Boolean range assertion.

Finally put $H(u) = C(N(u), G(u), u)$. The remaining apparent case split in $H(3u) = 3H(u)$ also has a purely equational replacement. Equation cases prove, for arbitrary parameters,

$$C(e, a+b, c+d) = C(e, a, c) + C(e, b, d), \qquad C(e, e, 0) = 0.$$

Using $N(3u) = N(u)$ and $G(3u) = 3G(u) + N(u)$, these yield

$$\begin{aligned} H(3u) &= C(N(u), 3G(u) + N(u), 3u) \\ &= 3C(N(u), G(u), u) + C(N(u), N(u), 0) = 3H(u). \end{aligned}$$

The equations $H(2u) = 3u$ and $H(1) = 1$ are immediate.

From this point the inherited comparator and saturation construction uses only equation induction and equality: the semigroup and rotation identities for iteration; $I(y, 3a; H) = 3I(y, a; H)$ and $I(y, 1; H) = 1$; the comparator $L(x, y) = N(I(y, I(x, 1; u.2u); H))$; and the saturated iterator $W(t, m) = I(t, 0; u.C(L(Su, m), u, Su))$. Its only subsequent numerical case, the identity $C(e, Sa, Sb) = S(C(e, a, b))$, is itself an equation case. The final finite scalar term

$$P(m) = I(m, 0; u.C(L(SSu, m), u, Su))$$

therefore satisfies

$$\mathcal{E} \vdash P(0) = 0, \qquad \mathcal{E} \vdash P(Sm) = m.$$

The whole derivation, not merely its conclusion, is stable under numerical substitution. Applying the already derived term P to $Sx = Sy$ gives $x = y$ in the same fragment.

B Internal scalar predecessor bootstrap

This scalar bootstrap is inherited from the supplied scalar development [6] and is reproduced to keep the scalar checker self-contained. It is not a new contribution of this paper. The proof uses a factor map fixing odd states and a saturated-iteration construction of predecessor.

The available arithmetic terms are 0, S, and

$$I(n, a; u.h), \qquad I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

The update binds only u. All other variables in it are fixed numerical parameters. We use equality reasoning, the already reconstructed zero/successor cases, and induction on a single literal equation. Every induction below keeps its other parameters fixed. Predecessor, successor injectivity, pairing, and induction on arbitrary quantifier-free formulas are not assumptions of the argument.

B.1 Elementary terms and two finite cycles

Write $1 = S0$ and use larger numerals as successor abbreviations. Define

$$C(n, a, b) = I(n, a; u.b), \qquad Z(n) = C(n, 1, 0), \qquad a + b = I(b, a; u.Su).$$

The update variable of C is absent from its body. Consequently

$$C(0,a,b) = a, \qquad C(Sn,a,b) = b.$$

Numerical cases show that $Z(n)$ is 0 or 1. The usual commutative-monoid equations for addition follow by literal equation induction. For example, $0 + b = b$ and $Sa + b = S(a + b)$ are proved by induction on b; these yield commutativity by a further equation induction, and associativity follows by induction on the last summand. We use these equations to reorder finite sums. The notation $2a$ means $a + a$ and $3a$ means $a + a + a$; no multiplication operation is needed for this bootstrap.

Put

$$N(n) = I(n, 0; u.Z(u)), \qquad j(u) = Z(u) + 2N(u), \qquad M(n) = I(n, 0; u.j(u)).$$

The range of N is contained in $\{0, 1\}$: its zero value is 0, and its successor value is a value of Z. This uses numerical cases on its counter, not induction on a disjunction. Hence

$$N(n) + Z(N(n)) = 1, \qquad Z(Z(N(n))) = N(n), \qquad N(n + 2) = N(n). \tag{4}$$

Here the last equation follows from two successor computations and the preceding equation. Literal-equation inductions give

$$N(2n) = 0, \qquad N(2n + 1) = 1, \qquad N(3n) = N(n). \tag{5}$$

For the first equation the successor step adds two to the argument of N; the second follows by one computation. In the third, increasing n adds three to the left argument and therefore applies Z once to both sides of the induction equation.

Finite computations now give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

The proof below needs just these three values and $M(Sn) = j(M(n))$; it does not assume a division algorithm or a theorem characterizing M as remainder modulo three.

B.2 A ramp with a global scaling identity

Define

$$G(n) = I(n, 0; u.u + M(u) + 1).$$

The central invariant is

$$M(G(n)) = N(n). \tag{6}$$

Its zero case is immediate. For its successor step, retain the single induction equation and split the Boolean value $N(n)$. If it is 0, the next G state is $G(n) + 1$; its M value is $j(0) = 1$, equal to $N(Sn)$. If it is 1, the next G state is $G(n) + 2$; two computations of M change its value from 1 to 2 and then to 0, again equal to $N(Sn)$. This proves the invariant by literal-equation induction.

It follows that

$$G(Sn) = G(n) + N(n) + 1, \qquad G(n + 2) = G(n) + 3.$$

Equation induction therefore yields

$$G(2n) = 3n. \tag{7}$$

A separate literal-equation induction proves

$$G(3n) = 3G(n) + N(n). \tag{8}$$

For completeness, its successor calculation is

$$G(3Sn) = G(3n) + N(3n) + 4 = 3G(n) + 2N(n) + 4,$$
$$3G(Sn) + N(Sn) = 3G(n) + 3N(n) + 3 + Z(N(n)).$$

The expressions agree in both Boolean cases for $N(n)$. The base is $0 = 0$. In particular, (8) was proved internally; it was not inferred from an intended floor-function interpretation of G.

Set

$$H(u) = C(N(u), G(u), u).$$

The crucial choice is to retain u when its parity is 1. The preceding equations give

$$H(2u) = 3u, \qquad H(1) = 1, \qquad H(3u) = 3H(u). \tag{9}$$

To check the last identity, use $N(3u) = N(u)$ and split the Boolean $N(u)$. At 0, both sides are $3G(u)$ by (8); at 1, both are $3u$. This argument does not require that an arbitrary number has already been represented as $2v$ or $2v + 1$.

B.3 A comparator from scalar iteration

For every fixed update h, equation induction on n, keeping m, a and all parameters fixed, proves the semigroup identity

$$I(m + n, a; h) = I(n, I(m, a; h); h).$$

The base is the zero computation and the step applies h to the induction equation. This identity will also be used in the later pairing construction.

For a fixed update h, equation induction on y, with its initializer a and all parameters of h fixed, proves the rotation identity

$$I(Sy, a; h) = I(y, h(a); h). \tag{10}$$

Both sides have base $h(a)$, and their successor equations apply the same h to the previous equality. Here and below $I(y, a; h)$ is an abbreviation for one fixed finite scalar term, not quantification over functions.

By (9), another equation induction gives

$$I(y, 3a; H) = 3I(y, a; H). \tag{11}$$

Its base is $3a = 3a$. In the step the fixed-parameter induction equation rewrites the new left side as

$$H(3I(y, a; H)) = 3H(I(y, a; H)).$$

The initializer parameter stays a throughout this induction. Equation induction also gives $I(y, 1; H) = 1$, since $H(1) = 1$.

Define

$$E(x) = I(x, 1; u.2u), \qquad L(x, y) = N\big(I(y, E(x); H)\big).$$

Then the following entire recursion specification is provable:

$$L(0, y) = 1, \qquad L(Sx, 0) = 0, \qquad L(Sx, Sy) = L(x, y). \tag{12}$$

The first equation uses $E(0) = 1$ and the fixed-point identity for H. The second uses $E(Sx) = 2E(x)$ and $N(2a) = 0$. For the third,

$$\begin{aligned} L(Sx, Sy) &= N(I(Sy, 2E(x); H)) \\ &= N(I(y, H(2E(x)); H)) \\ &= N(I(y, 3E(x); H)) \\ &= N(3I(y, E(x); H)) \\ &= N(I(y, E(x); H)) = L(x, y). \end{aligned}$$

The successive steps use rotation, $H(2a) = 3a$, iterator scaling, and $N(3a) = N(a)$. Thus L has the comparison equations at all arguments internally. Its standard-model meaning as the indicator of $x \leq y$ is motivation, not a premise of the following argument.

B.4 Saturated iteration gives predecessor

We now need only the three equations (12). Define

$$B_m(u) = C(L(Su, m), u, Su), \qquad W(t, m) = I(t, 0; u.B_m(u)).$$

The subscript in B_m records a fixed parameter; B_m is an open term abbreviation, not a function variable.

Numerical cases on e prove the selector equation

$$C(e, Sa, Sb) = SC(e, a, b).$$

Equivalently one may use equation induction on e, whose induction hypothesis is unused. Therefore

$$\begin{aligned} B_{Sm}(Su) &= C(L(SSu, Sm), Su, SSu) \\ &= C(L(Su, m), Su, SSu) = SB_m(u), \qquad (13) \\ B_{Sm}(0) &= C(L(S0, Sm), 0, S0) = C(L(0, m), 0, S0) = S0. \qquad (14) \end{aligned}$$

Literal-equation induction on t, with m fixed, proves

$$W(St, Sm) = SW(t, m). \qquad (15)$$

The base is (14). The successor step is

$$\begin{aligned} W(SSt, Sm) &= B_{Sm}(W(St, Sm)) \\ &= B_{Sm}(SW(t, m)) \\ &= SB_m(W(t, m)) = SW(St, m). \end{aligned}$$

Only the second line uses the current induction hypothesis, at the same fixed m.

Two subsequent equation inductions give

$$L(Sn, n) = 0, \qquad W(n, n) = n. \qquad (16)$$

The first uses the comparator recurrence. The second uses (15) in its successor step. The substitution $t = m = n$ is made into that already proved universally parameterized equation; no parameter of an active induction hypothesis is changed. Consequently

$$W(Sn, n) = B_n(n) = C(L(Sn, n), n, Sn) = n. \qquad (17)$$

Finally define the finite scalar term

$$P(m) = I\big(m, 0; u.C(L(SSu, m), u, Su)\big). \tag{18}$$

Its zero equation is immediate. To justify its successor equation without assuming a separate extensionality rule for iterator bodies, put

$$V(t, m) = I\big(t, 0; u.C(L(SSu, Sm), u, Su)\big).$$

The comparator recurrence identifies this update pointwise with $B_m(u)$. Equation induction on t, with m fixed, gives $V(t, m) = W(t, m)$: their base values are 0, and the successor step uses the pointwise update equation and then the induction equation. Now (17) yields

$$\boxed{P(0) = 0, \qquad P(Sm) = V(Sm, m) = W(Sm, m) = m.}$$

B.5 Dependency boundary

All terms in this section are built from $0, S, I$ by finite substitution. All inductions have a single equation as their invariant. Numerical cases and equality transport are the already reconstructed logical resources. Successor injectivity may now be obtained by applying P to an assumed equation $Sx = Sy$. Equality tests, reflection of quantifier-free formulas, pairing, and the recovery of general primitive recursion can then be developed in the established order. They were not used in order to manufacture predecessor.

This argument is a written internal derivation in the stated theory. The finite implementation checks do not constitute a proof-assistant formalization of this entire metatheorem.

References

[1] V. Reshetnikov and ProveIt contributors, *Concise presentations of intuitionistic primitive recursive arithmetic*, README at commit `8fa5ade009`; the linked source records the full identifier. Pinned README and package inventory.

[2] ProveIt contributors, *Native.lean and NativeEq.lean*, same revision. Native syntax and eight-rule kernel.

[3] ProveIt contributors, *NativeBiInterpretation.lean*, same revision. Faithfulness, both formula round trips, and equiconsistency.

[4] ProveIt contributors, *NativeWitnessShape.lean*, same revision. The two witness-shape obstructions and the open combined question.

[5] ProveIt contributors, *Complementary introductions with one local assumption*, inherited reference in `Research/certified_zero`, same revision. Logical macros and native round trips.

[6] ProveIt contributors, *Internal scalar predecessor bootstrap*, `Research/certified_zero/reference/scalar_predecessor.tex`, same revision. Reproduced in the final appendix with attribution; the preceding appendix audits its witness-free dependency boundary. Pinned scalar construction.

[7] ProveIt contributors, *Deterministic introduction*, `Research/deterministic_introduction`, same revision. Matched control and previous naming-pattern checker.